PRAKTIKUM PENGEMBANGAN APLIKASI WEB

MODUL 2

HTML Semantik, Tailwind CSS, dan Aksesibilitas



Penulis: Muhammad Fakhrul Safitra, S.Kom., M.Kom.
Kode MK 52357 · 1 sks · Semester 5 · September 2026

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN ILMU KOMPUTER · UNIVERSITAS PERTAMINA

Hak cipta © 2026 Universitas Pertamina. Dilarang memperbanyak sebagian atau seluruh isi modul tanpa izin tertulis.

Identitas Modul

Mata Kuliah	Praktikum Pengembangan Aplikasi Web (52357), 1 sks, semester 5
Mata Kuliah Pendamping	Pengembangan Aplikasi Web (52355), pekan 2: HTML semantik, CSS responsif, dan Tailwind CSS
Modul / Pekan	Modul 2 / Pekan 2
Sub-CPMK	Sub-CPMK 2: Mahasiswa mampu mengimplementasikan antarmuka web yang responsif dan aksesibel menggunakan HTML semantik, Tailwind CSS, TypeScript, dan komponen React pada produknya, dengan tampilan yang berfungsi pada tiga ukuran layar dan skor aksesibilitas Lighthouse minimal 85 (C3, P3).
Alokasi Waktu	170 menit: Tugas Pendahuluan 20 menit (sebelum sesi), praktikum laboratorium 110 menit termasuk Tes 20 menit, dan Dokumen Teknis 40 menit (setelah sesi)
Bobot Penilaian	Bersama Modul 3 membentuk bobot Sub-CPMK 2: Tugas Pendahuluan 2%, Tes 3%, dan Dokumen Teknis 5% dari nilai akhir praktikum; nilai Modul 2 dan Modul 3 dirata-ratakan untuk setiap komponen
Luaran	Halaman utama produk yang semantik, responsif pada tiga ukuran layar, dan memperoleh skor aksesibilitas Lighthouse minimal 85; *pull request* `praktikum/modul-02` yang telah digabungkan; berkas `docs/praktikum/modul-02.md`

A. Capaian Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. menyusun struktur halaman menggunakan elemen semantik HTML sehingga setiap bagian halaman memiliki peran *landmark* yang jelas;
2. menerapkan Flexbox dan Grid melalui kelas utilitas Tailwind CSS untuk menata letak komponen halaman;
3. menerapkan pendekatan *mobile-first* dengan *breakpoint* Tailwind CSS sehingga tampilan berfungsi pada tiga ukuran layar;
4. membangun formulir yang aksesibel dengan label, petunjuk, dan penanda fokus yang tepat;
5. melakukan audit aksesibilitas menggunakan Lighthouse serta memperbaiki temuan audit dan hasil pemeriksaan manual dengan papan ketik.

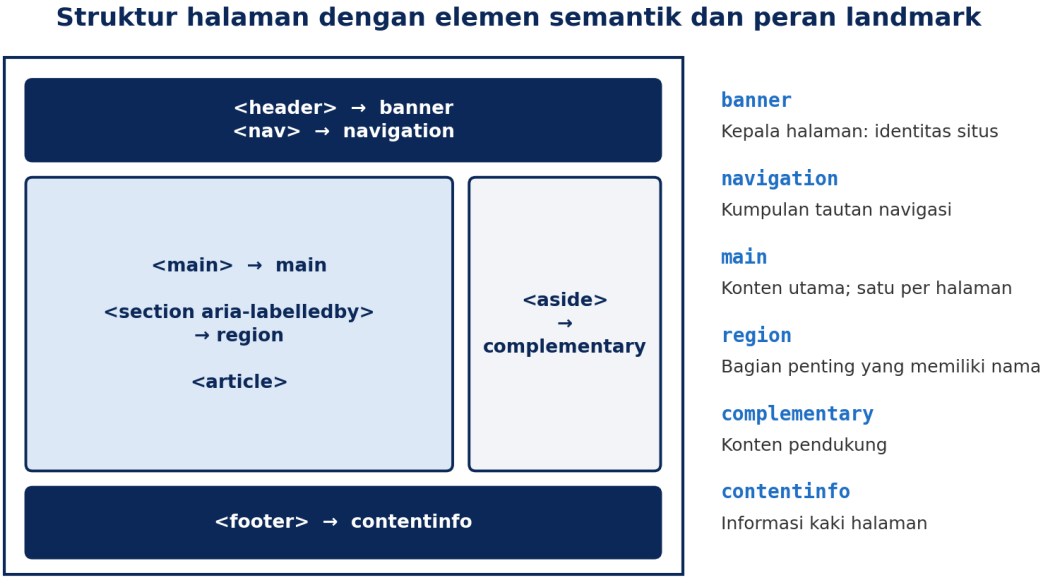
Indikator keberhasilan: halaman utama produk berfungsi tanpa gulir horizontal pada lebar 360 px, 768 px, dan 1280 px, memperoleh skor aksesibilitas Lighthouse minimal 85, dan seluruh elemen interaktif dapat dijangkau dengan tombol Tab.

B. Dasar Teori

1. HTML Semantik dan *Landmark*

HTML semantik adalah penggunaan elemen yang maknanya sesuai dengan isi, misalnya `<nav>` untuk kumpulan tautan navigasi dan `<main>` untuk konten utama, alih-alih `<div>` yang tidak bermakna [2]. Makna tersebut dibaca oleh peramban, mesin pencari, dan teknologi bantu seperti pembaca layar. Beberapa elemen semantik dipetakan ke **peran *landmark*** ARIA

sehingga pengguna pembaca layar dapat berpindah langsung ke bagian tertentu halaman [5], [6].



Gambar 1. *Pemetaan elemen semantik ke peran landmark*

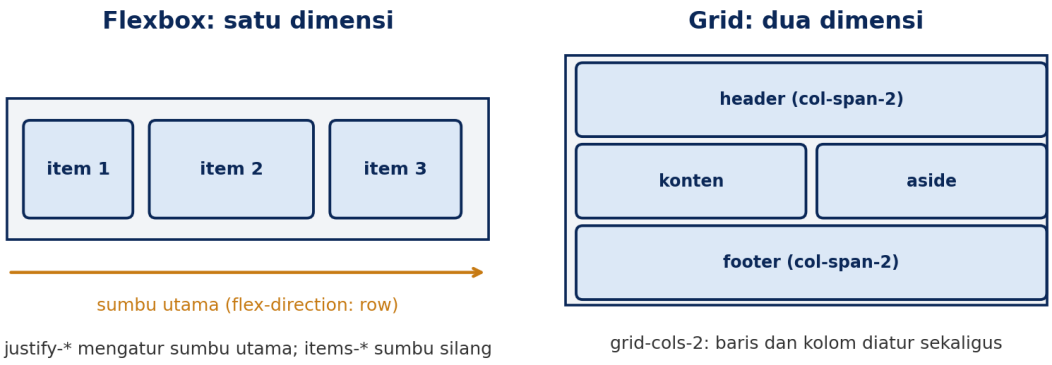
Pemetaan pada Gambar 1 berlaku dengan beberapa syarat. Elemen `<header>` dan `<footer>` hanya berperan sebagai banner dan contentinfo apabila tidak berada di dalam `<article>`, `<aside>`, `<main>`, `<nav>`, atau `<section>`. Elemen `<section>` baru menjadi *landmark* region apabila memiliki nama yang dapat diakses, misalnya melalui atribut `aria-labelledby` yang merujuk ke judul bagian. Setiap halaman sebaiknya memiliki tepat satu `<main>` dan hierarki judul yang runtut dari `<h1>` hingga tingkat berikutnya tanpa melompat.

HTML di dalam JSX

Halaman Next.js ditulis dalam JSX, yaitu sintaksis serupa HTML di dalam TypeScript. Beberapa atribut ditulis berbeda karena merupakan kata kunci JavaScript: `class` menjadi `className` dan `for` menjadi `htmlFor` [9]. Setiap elemen harus ditutup, termasuk elemen kosong seperti `<img />` dan `<input />`.

2. Tata Letak dengan Flexbox dan Grid

Flexbox menata elemen dalam satu dimensi, yaitu baris atau kolom. Kelas `justify-*` mengatur distribusi elemen pada sumbu utama, sedangkan `items-*` mengatur perataan pada sumbu silang [3]. **Grid** menata elemen dalam dua dimensi sekaligus, yaitu baris dan kolom, sehingga sesuai untuk tata letak halaman dan kumpulan kartu [3]. Gambar 2 membandingkan kedua pendekatan.



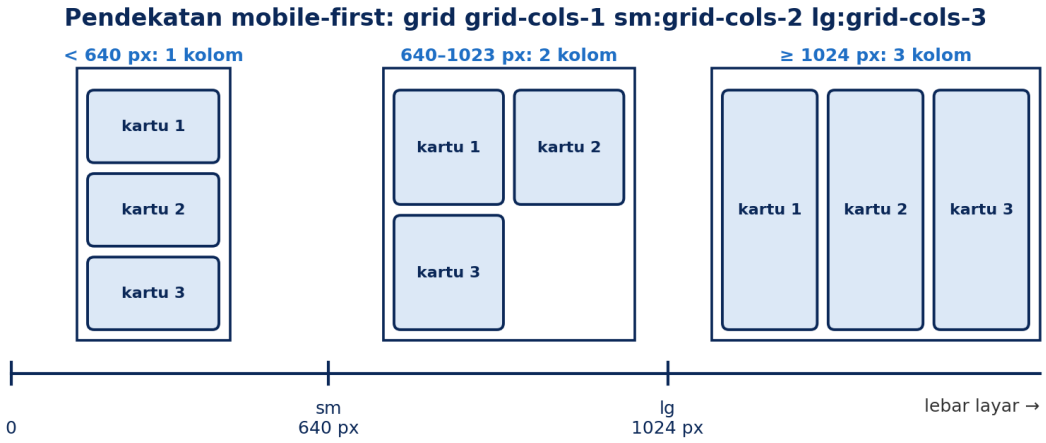
Gambar 2. *Perbandingan Flexbox dan Grid*

Tabel 1. Kelas utilitas Tailwind CSS untuk tata letak

Kelas	Arti
`flex`, `flex-col`, `flex-row`	Mengaktifkan Flexbox dan menentukan arah sumbu utama
`justify-between`, `items-center`	Membagi ruang di antara elemen; meratakan elemen di tengah sumbu silang
`flex-wrap`, `gap-4`	Memindahkan elemen ke baris baru bila ruang tidak cukup; memberi jarak 1rem antarelemen
`grid`, `grid-cols-3`	Mengaktifkan Grid dengan tiga kolom sama lebar
`col-span-2`	Membuat elemen menempati dua kolom
`grid-cols-[2fr_1fr]`	Nilai sembarang (arbitrary value): kolom pertama dua kali lebar kolom kedua
`max-w-6xl`, `mx-auto`, `w-full`	Membatasi lebar maksimum dan meletakkan konten di tengah

3. Desain Responsif dengan Pendekatan *Mobile-First*

Tailwind CSS versi 4 menerapkan pendekatan *mobile-first*: kelas tanpa awalan berlaku untuk semua ukuran layar, sedangkan kelas berawalan *breakpoint* seperti `md:` berlaku mulai lebar tersebut ke atas [4]. *Breakpoint* bawaan adalah `sm` (40rem atau 640 px), `md` (48rem atau 768 px), `lg` (64rem atau 1024 px), `xl` (80rem atau 1280 px), dan `2xl` (96rem atau 1536 px). Dengan demikian, tampilan ponsel ditulis lebih dahulu tanpa awalan, lalu disesuaikan untuk layar yang lebih lebar, seperti pada Gambar 3.



Gambar 3. *Perubahan jumlah kolom pada pendekatan mobile-first*

Proyek yang dibuat dengan `create-next-app` telah memuat Tailwind CSS melalui baris `@import "tailwindcss";` pada berkas `app/globals.css`. Nilai tema seperti warna merek ditambahkan sebagai variabel pada blok `@theme`, misalnya `--color-brand: #0b2858;`, yang kemudian dapat dipakai sebagai kelas `bg-brand` atau `text-brand` [4].

4. Aksesibilitas Web dan Audit Lighthouse

Aksesibilitas web adalah kemampuan aplikasi untuk digunakan oleh semua orang, termasuk pengguna dengan disabilitas penglihatan, pendengaran, motorik, atau kognitif. Acuan yang digunakan adalah Web Content Accessibility Guidelines (WCAG) 2.2 [5]. Tabel 2 merangkum kriteria yang paling sering dijumpai pada halaman produk.

Tabel 2. Kriteria aksesibilitas yang diperiksa pada modul ini

Kriteria WCAG 2.2	Syarat	Penerapan
1.1.1 Konten Nonteks	Gambar informatif memiliki teks alternatif	`alt` deskriptif; `alt=""` untuk gambar dekoratif
1.3.1 Informasi dan Relasi	Struktur dapat dikenali secara terprogram	Elemen semantik, hierarki judul, `<label>`
1.4.3 Kontras Minimum	Rasio kontras teks minimal 4,5:1 (teks besar 3:1)	Hindari teks abu-abu muda di atas latar putih
2.1.1 Papan Ketik	Seluruh fungsi dapat dioperasikan dengan papan ketik	Gunakan `<button>` dan `<a>`, bukan `<div>` yang dapat diklik
2.4.7 Fokus Terlihat	Posisi fokus papan ketik terlihat jelas	Kelas `focus-visible:outline-*`
4.1.2 Nama, Peran, Nilai	Komponen memiliki nama yang dapat diakses	`aria-label` pada tombol ikon

Lighthouse adalah alat audit pada Chrome DevTools. Skor aksesibilitas Lighthouse merupakan rata-rata berbobot dari audit otomatis berbasis axe-core; setiap audit bernilai lulus atau gagal tanpa nilai parsial [7]. Skor 100 hanya berarti seluruh audit otomatis lulus. Sebagian masalah, seperti urutan fokus yang tidak logis atau teks alternatif yang tidak bermakna, hanya dapat ditemukan melalui pemeriksaan manual. Oleh karena itu, audit Lighthouse pada modul ini selalu dilengkapi pemeriksaan dengan papan ketik.

C. Alat dan Bahan

- Lingkungan pengembangan dan repositori produk dari Modul 1.
- Peramban Google Chrome atau Microsoft Edge versi terbaru dengan panel Lighthouse pada DevTools.
- Ekstensi Visual Studio Code Tailwind CSS IntelliSense untuk saran dan pratinjau kelas.
- Sketsa (*wireframe*) halaman utama produk dari Tugas Pendahuluan.

D. Persiapan

> **Catatan**
> Persiapan berikut dilakukan sebelum sesi praktikum dan memerlukan waktu sekitar 5 menit.

1. Pastikan *branch* `main` di komputer sama dengan GitHub, lalu buat *branch* kerja modul ini:

```
Terminal
git switch main
git pull
git switch -c praktikum/modul-02
npm run dev
```

2. Siapkan sketsa halaman utama untuk tampilan ponsel dan desktop, lengkap dengan penanda elemen semantik setiap bagian (Tugas Pendahuluan nomor 1).
3. Tutup ekstensi peramban yang tidak diperlukan, atau siapkan jendela Incognito/InPrivate, agar ekstensi tidak memengaruhi hasil audit Lighthouse.

E. Langkah Praktikum

Praktikum dilaksanakan dalam lima bagian dengan pembagian waktu pada Tabel 3. Setiap bagian diakhiri titik periksa (*checkpoint*) yang diperiksa oleh asisten atau dosen. Seluruh pekerjaan dilakukan pada halaman utama produk masing-masing; teks contoh pada modul diganti dengan isi yang sesuai dengan produk.

Tabel 3. Pembagian waktu sesi laboratorium

Bagian	Kegiatan	Waktu
1	Struktur semantik halaman	15 menit
2	Tata letak dengan Flexbox dan Grid	25 menit
3	Penyesuaian responsif	15 menit
4	Formulir yang aksesibel	15 menit
5	Audit aksesibilitas dan perbaikan	20 menit
–	Tes	20 menit

Bagian 1 — Struktur Semantik Halaman

1. Buka `app/layout.tsx`. Ubah bahasa dokumen menjadi bahasa Indonesia dan isi metadata halaman:

```
app/layout.tsx (bagian yang diubah)
export const metadata: Metadata = {
  title: "Nama Produk",
  description: "Deskripsi singkat produk dalam satu kalimat.",
};

// ... di dalam RootLayout
<html lang="id">
```

Atribut `lang` menentukan cara pembaca layar melafalkan teks, sedangkan `title` tampil pada tab peramban dan hasil mesin pencari [10].

2. Ganti seluruh isi `app/page.tsx` dengan kerangka semantik berikut:

```
app/page.tsx
import Link from "next/link";

export default function Beranda() {
  return (
    <>
      <a href="#konten" className="sr-only focus:not-sr-only focus:p-2">
        Lewati ke konten utama
      </a>
      <header className="border-b">
        <nav aria-label="Navigasi utama">
          <Link href="/">NamaProduk</Link>
          <ul>
            <li><a href="#fitur">Fitur</a></li>
            <li><a href="#kontak">Kontak</a></li>
          </ul>
        </nav>
      </header>

      <main id="konten">
        <section aria-labelledby="judul-utama">
          <h1 id="judul-utama">Kalimat nilai utama produk</h1>
          <p>Penjelasan singkat permasalahan dan solusi produk.</p>
        </section>

        <section id="fitur" aria-labelledby="judul-fitur">
          <h2 id="judul-fitur">Fitur Utama</h2>
          {/* kartu fitur ditambahkan pada Bagian 2 */}
        </section>

        <section id="kontak" aria-labelledby="judul-kontak">
          <h2 id="judul-kontak">Hubungi Kami</h2>
          {/* formulir ditambahkan pada Bagian 4 */}
        </section>
      </main>

      <footer className="border-t">
        <p>© 2026 Nama Produk</p>
      </footer>
    </>
  );
}
```

Tautan "Lewati ke konten utama" disembunyikan secara visual dengan kelas `sr-only` dan baru tampil ketika menerima fokus papan ketik. Tautan ini memungkinkan pengguna papan ketik melewati navigasi. Komponen `Link` dari `next/link` digunakan untuk tautan ke halaman lain di dalam aplikasi, sedangkan tautan ke bagian pada halaman yang sama cukup menggunakan `<a href="#...">`.

3. Periksa struktur halaman melalui DevTools: buka panel **Elements**, lalu tab **Accessibility**, dan aktifkan *Enable full-page accessibility tree*. Pastikan *landmark* banner, navigation, main, region, dan contentinfo muncul pada pohon aksesibilitas [8].

> ***Checkpoint* 1**
> Halaman memiliki satu `<h1>`, judul `<h2>` untuk setiap section, satu `<main>`, dan *landmark* yang lengkap pada pohon aksesibilitas.

Bagian 2 — Tata Letak dengan Flexbox dan Grid

1. Tata navigasi dengan Flexbox. Ubah elemen `<nav>` dan `<ul>` pada `app/page.tsx`:

app/page.tsx (navigasi)

```
<nav aria-label="Navigasi utama"
     className="mx-auto flex max-w-6xl items-center justify-between p-4">
  <Link href="/" className="text-lg font-bold">NamaProduk</Link>
  <ul className="flex gap-6">
    ...
  </ul>
</nav>
```

2. Tampilkan kartu fitur dengan Grid. Tambahkan data fitur di atas fungsi `Beranda`, lalu isi section fitur:

app/page.tsx (kartu fitur)

```
const fitur = [
  { judul: "Fitur pertama", deskripsi: "Manfaat fitur bagi pengguna." },
  { judul: "Fitur kedua", deskripsi: "Manfaat fitur bagi pengguna." },
  { judul: "Fitur ketiga", deskripsi: "Manfaat fitur bagi pengguna." },
];

// di dalam <section id="fitur">
<ul className="mt-6 grid grid-cols-3 gap-6">
  {fitur.map((f) => (
    <li key={f.judul}>
      <article className="h-full rounded-lg border p-6">
        <h3 className="text-lg font-semibold">{f.judul}</h3>
        <p className="mt-2 text-gray-700">{f.deskripsi}</p>
      </article>
    </li>
  ))}
</ul>
```

Ekspresi `fitur.map(...)` menghasilkan satu kartu untuk setiap data dan dibahas lebih lanjut pada Modul 3. Atribut `key` diperlukan agar React dapat mengenali setiap elemen daftar.

3. Bungkus isi `<main>` dengan kelas `mx-auto max-w-6xl p-4` agar konten tidak melebar pada layar besar.
4. Tambahkan tata letak dua kolom untuk konten dan informasi pendukung menggunakan Grid dengan nilai sembarang:

app/page.tsx (konten dan aside)

```
<div className="grid gap-8 grid-cols-[2fr_1fr]">
  <section aria-labelledby="judul-cara">
    <h2 id="judul-cara">Cara Kerja</h2>
    ...
  </section>
  <aside aria-label="Informasi tambahan"
         className="rounded-lg bg-gray-100 p-6">
    ...
  </aside>
</div>
```

Checkpoint 2

Navigasi tersusun mendatar dengan logo di kiri dan menu di kanan, kartu fitur tampil dalam tiga kolom, dan bagian konten serta aside tampil berdampingan pada layar desktop.

Bagian 3 — Penyesuaian Responsif

1. Aktifkan *Device Toolbar* pada DevTools dengan menekan Ctrl + Shift + M (Windows/Linux) atau Cmd + Shift + M (macOS), lalu atur lebar 360 px [8]. Amati bahwa kartu tiga kolom dan tata letak dua kolom menjadi terlalu sempit atau memunculkan gulir horizontal.
2. Terapkan pendekatan *mobile-first*: tulis tampilan ponsel tanpa awalan, lalu tambahkan awalan *breakpoint* untuk layar yang lebih lebar.

app/page.tsx (kelas yang diubah)

```
{/* navigasi: bertumpuk di ponsel, mendatar mulai 640 px */}
<nav className="mx-auto flex max-w-6xl flex-col gap-3 p-4
                sm:flex-row sm:items-center sm:justify-between">
  <ul className="flex flex-col gap-2 sm:flex-row sm:gap-6">

{/* kartu: 1 kolom, 2 kolom mulai 640 px, 3 kolom mulai 1024 px */}
<ul className="mt-6 grid grid-cols-1 gap-6 sm:grid-cols-2 lg:grid-cols-3">

{/* konten dan aside: bertumpuk, berdampingan mulai 1024 px */}
<div className="grid gap-8 lg:grid-cols-[2fr_1fr]">
```

3. Periksa tampilan pada lebar 360 px, 768 px, dan 1280 px. Pastikan tidak ada gulir horizontal, teks tetap terbaca, dan tautan tetap mudah disentuh.
4. Tambahkan warna merek pada blok `@theme` di `app/globals.css`, misalnya `--color-brand: #0b2858;`, lalu gunakan kelas `bg-brand` atau `text-brand` pada *header* atau judul. Pastikan kontras teks terhadap latar tetap memenuhi rasio 4,5:1.
5. Ambil tangkapan layar pada ketiga lebar tersebut untuk Dokumen Teknis. Gunakan menu titik tiga pada Device Toolbar, lalu pilih *Capture screenshot*.

Checkpoint 3

Halaman berfungsi tanpa gulir horizontal pada lebar 360 px, 768 px, dan 1280 px, dan jumlah kolom kartu berubah sesuai *breakpoint*.

Bagian 4 — Formulir yang Aksesibel

1. Isi section kontak dengan formulir berikut. Pemrosesan data formulir di server dibahas pada Modul 5; pada modul ini fokusnya adalah struktur dan aksesibilitas.

app/page.tsx (formulir kontak)

```
// di atas fungsi Beranda: kelas bersama untuk kolom isian
const kolom =
  "rounded border px-3 py-2 focus-visible:outline-2 " +
  "focus-visible:outline-offset-2 focus-visible:outline-blue-700";

// di dalam <section id="kontak">
<form className="mt-4 grid max-w-xl gap-4">
  <div className="flex flex-col gap-1">
    <label htmlFor="nama" className="font-medium">Nama lengkap</label>
    <input id="nama" name="nama" type="text" required
      autoComplete="name" className={kolom} />
  </div>

  <div className="flex flex-col gap-1">
    <label htmlFor="email" className="font-medium">Surel</label>
    <input id="email" name="email" type="email" required
      autoComplete="email" aria-describedby="email-bantuan"
      className={kolom} />
    <p id="email-bantuan" className="text-sm text-gray-600">
      Gunakan alamat surel yang aktif.
    </p>
  </div>

  <fieldset className="flex flex-col gap-1">
    <legend className="font-medium">Peran</legend>
    <label>
      <input type="radio" name="peran" value="pengguna" /> Pengguna
    </label>
    <label>
      <input type="radio" name="peran" value="mitra" /> Mitra
    </label>
  </fieldset>

  <div className="flex flex-col gap-1">
    <label htmlFor="pesan" className="font-medium">Pesan</label>
    <textarea id="pesan" name="pesan" rows={4} className={kolom} />
  </div>

  <button type="submit"
    className={kolom + " bg-blue-700 font-semibold text-white"}>
    Kirim
  </button>
</form>
```

Tabel 4. Unsur aksesibilitas pada formulir

Unsur	Fungsi
`kolom` (konstanta kelas)	Menyimpan kelas yang sama untuk seluruh kolom isian agar tidak ditulis berulang
`<label htmlFor>`	Menghubungkan label dengan kolom isian; mengeklik label memindahkan fokus ke kolom
`aria-describedby`	Membacakan teks petunjuk setelah label ketika kolom menerima fokus
`<fieldset>` dan `<legend>`	Mengelompokkan pilihan radio dan memberi nama kelompok
`autoComplete`	Membantu peramban mengisi data yang umum, misalnya nama dan surel
`focus-visible:outline-*`	Menampilkan garis fokus yang jelas saat navigasi dengan papan ketik

2. Uji formulir hanya dengan papan ketik: tekan Tab untuk berpindah, Shift + Tab untuk kembali, Spasi untuk memilih radio, dan Enter untuk mengirim. Pastikan urutan fokus mengikuti urutan visual dan garis fokus selalu terlihat.

Checkpoint 4

Setiap kolom memiliki label yang terhubung, pilihan radio dikelompokkan dengan legend, dan seluruh formulir dapat dioperasikan dengan papan ketik.

Bagian 5 — Audit Aksesibilitas dan Perbaikan

1. Buat halaman latihan yang sengaja memuat beberapa masalah aksesibilitas pada berkas `app/latihan-audit/page.tsx`:

app/latihan-audit/page.tsx

```
export default function LatihanAudit() {
  return (
    <div className="p-8">
      <div className="text-2xl font-bold">Katalog Alat Laboratorium</div>
      <img src="/next.svg" width={120} height={24} />
      <p className="text-gray-300">Stok diperbarui setiap hari.</p>
      <input type="search" className="border p-2" />
      <button className="ml-2 border p-2">
        <svg width="16" height="16" viewBox="0 0 16 16">
          <circle cx="7" cy="7" r="5" stroke="currentColor" fill="none" />
        </svg>
      </button>
    </div>
  );
}
```

2. Buka http://localhost:3000/latihan-audit, lalu buka panel **Lighthouse** pada DevTools. Pilih mode *Navigation*, perangkat *Mobile*, dan kategori *Accessibility* saja, kemudian klik *Analyze page load*.
3. Catat skor dan setiap audit yang gagal. Klik setiap audit untuk melihat elemen yang bermasalah.

4. Perbaiki halaman latihan hingga skor aksesibilitas minimal 90. Tabel 5 memuat petunjuk perbaikan.

Tabel 5. Temuan audit pada halaman latihan dan arah perbaikannya

Audit yang gagal	Arah perbaikan
Image elements do not have [alt] attributes	Tambahkan `alt` yang menjelaskan isi gambar, atau `alt=""` bila gambar hanya dekoratif
Background and foreground colors do not have a sufficient contrast ratio	Ganti `text-gray-300` dengan warna yang lebih gelap, misalnya `text-gray-700`
Form elements do not have associated labels	Tambahkan `<label htmlFor>` yang terlihat, misalnya "Cari alat"
Buttons do not have an accessible name	Tambahkan `aria-label="Cari"` pada tombol dan `aria-hidden="true"` pada ikon SVG

Selain empat temuan tersebut, judul halaman yang ditulis dengan `<div>` perlu diganti dengan `<h1>`. Masalah ini tidak tercatat sebagai audit gagal oleh Lighthouse, tetapi terlihat pada pemeriksaan manual pohon aksesibilitas. Hal serupa berlaku untuk *placeholder*: alat audit otomatis dapat menerima *placeholder* sebagai nama kolom, padahal teks tersebut hilang ketika pengguna mulai mengetik dan sering berkontras rendah, sehingga label yang terlihat tetap diperlukan.

5. Jalankan audit yang sama pada halaman utama produk (http://localhost:3000). Perbaiki temuan hingga skor minimal 85, lalu lakukan pemeriksaan manual dengan tombol Tab.
6. Hapus folder `app/latihan-audit` setelah hasil audit dicatat, karena halaman tersebut bukan bagian dari produk.
7. Simpan pekerjaan, kirim *branch*, dan gabungkan melalui *pull request* seperti pada Modul 1:

```
Terminal
git add .
git commit -m "feat: halaman utama semantik, responsif, dan aksesibel"
git push -u origin praktikum/modul-02
```

> ***Checkpoint* 5**
>
> Skor aksesibilitas halaman latihan setelah perbaikan minimal 90 dan skor halaman utama produk minimal 85.
>
> *Pull request* `praktikum/modul-02` telah digabungkan ke `main`.

F. Tugas Pendahuluan

Waktu: 20 menit, dikumpulkan melalui e-learning sebelum sesi praktikum sesuai jadwal pada e-learning. **Bobot:** bagian dari 2% Sub-CPMK 2 (rerata Modul 2 dan Modul 3).

1. Unggah sketsa halaman utama produk untuk tampilan ponsel dan desktop. Beri penanda elemen semantik pada setiap bagian, misalnya *header*, nav, main, section, aside, dan footer.

2. Jelaskan perbedaan Flexbox dan Grid, lalu berikan satu contoh bagian halaman produk Anda yang tepat untuk masing-masing.
3. Jelaskan makna pendekatan *mobile-first* pada Tailwind CSS dan arti kelas `grid-cols-1 md:grid-cols-2`.
4. Jelaskan fungsi atribut `alt`, serta kapan atribut tersebut diisi deskripsi dan kapan dikosongkan.
5. Jelaskan alasan *placeholder* tidak dapat menggantikan label pada kolom formulir.

Penilaian menggunakan rubrik untuk nomor 1 (kelengkapan bagian dan ketepatan elemen semantik) dan kunci jawaban untuk nomor 2–5.

G. Tes

Waktu: 20 menit terakhir sesi laboratorium, dikerjakan secara individual melalui e-learning tanpa bantuan kecerdasan artifisial. **Bobot:** bagian dari 3% Sub-CPMK 2 (rerata Modul 2 dan Modul 3).

Tes terdiri atas 10 soal yang mengukur kemampuan:

- memprediksi tampilan dari kombinasi kelas Flexbox, Grid, dan *breakpoint* Tailwind CSS;
- mengenali peran *landmark* dan elemen semantik yang tepat;
- menentukan perbaikan atas temuan audit aksesibilitas pada potongan kode.

Penilaian menggunakan kunci jawaban.

H. Dokumen Teknis

Waktu: 40 menit setelah sesi, paling lambat tiga hari setelah praktikum. **Bobot:** bagian dari 5% Sub-CPMK 2 (rerata Modul 2 dan Modul 3).

Dokumen Teknis disimpan pada berkas `docs/praktikum/modul-02.md` di repositori produk, di-commit, lalu di-push. Tautan berkas tersebut di GitHub dikumpulkan melalui e-learning. Gunakan kerangka berikut.

docs/praktikum/modul-02.md

```
# Dokumen Teknis Modul 2 — HTML Semantik, Tailwind CSS, dan Aksesibilitas

Nama/NIM   :
Repositori :

## 1. Struktur Semantik
- Kerangka landmark dan hierarki judul halaman utama
- Tangkapan layar pohon aksesibilitas pada DevTools

## 2. Tata Letak Responsif
- Tangkapan layar pada lebar 360 px, 768 px, dan 1280 px
- Kelas Flexbox, Grid, dan breakpoint yang digunakan beserta alasannya

## 3. Audit Aksesibilitas
- Tabel skor Lighthouse sebelum dan sesudah perbaikan
  (halaman latihan dan halaman utama)
- Daftar audit yang gagal, penyebab, dan perbaikannya
- Hasil pemeriksaan manual dengan papan ketik
```

```
  (urutan fokus dan garis fokus)

## 4. Kendala dan Penyelesaian

## 5. Catatan Pemanfaatan AI
Alat, perintah utama, bagian yang digunakan, dan cara memverifikasinya.
Tulis "Tidak menggunakan AI" apabila tidak menggunakan AI.
```

Tabel 6. Rubrik Dokumen Teknis

Kriteria	Bobot	4 (Sangat Baik)	3 (Baik)	2 (Cukup)	1 (Kurang)
Ketepatan teknis	35%	Struktur semantik, kelas tata letak, dan perbaikan aksesibilitas seluruhnya tepat	Terdapat satu kekeliruan kecil	Terdapat beberapa kekeliruan	Banyak kekeliruan mendasar
Bukti pengujian	30%	Tangkapan layar tiga ukuran, skor sebelum dan sesudah, serta hasil uji papan ketik lengkap	Satu bukti belum lengkap	Sebagian besar bukti belum lengkap	Bukti tidak tersedia
Penjelasan keputusan teknis	20%	Alasan pemilihan Flexbox atau Grid dan setiap perbaikan dijelaskan runtut	Alasan tepat tetapi kurang mendalam	Penjelasan bersifat deskriptif	Tidak ada penjelasan
Pencatatan pemanfaatan AI	15%	Lengkap, spesifik, dan disertai cara verifikasi	Lengkap tanpa cara verifikasi	Sangat umum	Tidak dicantumkan

Nilai Dokumen Teknis = Σ (bobot kriteria × skor/4) × 100.

I. Keterkaitan dengan MK Pengembangan Aplikasi Web

- Halaman utama yang dibangun pada modul ini menjadi dasar Tugas 1 (Antarmuka Produk, penanda versi `v0.1`, pekan 3), yang mensyaratkan responsivitas pada tiga ukuran layar dan skor aksesibilitas Lighthouse minimal 85.
- Latihan 1 pada MK Pengembangan Aplikasi Web menilai revisi tata letak hasil asisten kecerdasan artifisial. Praktikum ini menilai implementasi dan audit yang dikerjakan langsung, sehingga kedua penilaian tidak tumpang tindih.
- Pemrosesan formulir di server dibahas pada Modul 5, sedangkan pemecahan halaman menjadi komponen React dibahas pada Modul 3.

J. Troubleshooting

Masalah	Penyebab	Penyelesaian
Kelas Tailwind tidak berpengaruh	Salah ketik nama kelas, atau baris `@import "tailwindcss";` terhapus dari `app/globals.css`	Periksa saran Tailwind CSS IntelliSense dan pastikan `globals.css` diimpor pada `app/layout.tsx`
Kelas yang disusun dari potongan string tidak berlaku, misalnya `` `grid-cols-${n}` ``	Tailwind hanya membuat kelas yang tertulis utuh pada kode sumber	Tulis nama kelas secara lengkap, misalnya melalui objek pemetaan nilai ke kelas
Peringatan *Invalid DOM property `class` atau `for`*	Menggunakan atribut HTML di dalam JSX	Gunakan `className` dan `htmlFor`
Peringatan ESLint `@next/next/no-img-element`	Penggunaan `<img>` alih-alih komponen `Image` Next.js	Pada halaman latihan peringatan dapat diabaikan; pada produk gunakan `Image` dari `next/image`
Muncul gulir horizontal pada ponsel	Lebar tetap (misalnya `w-[800px]`) atau tata letak multikolom tanpa awalan *breakpoint*	Gunakan `w-full` dan `max-w-*`, dan terapkan kelas multikolom dengan awalan `sm:`, `md:`, atau `lg:`
Skor Lighthouse berubah-ubah atau audit gagal dijalankan	Pengaruh ekstensi peramban	Jalankan audit pada jendela Incognito/InPrivate
Panel Lighthouse tidak terlihat	Panel tersembunyi di menu DevTools	Klik ikon » pada deretan panel DevTools lalu pilih Lighthouse

K. Ketentuan Umum

- Asisten kecerdasan artifisial diperkenankan pada Tugas Pendahuluan dan Dokumen Teknis dengan kewajiban mencatat penggunaannya. Tes dikerjakan tanpa bantuan kecerdasan artifisial.
- Setiap mahasiswa wajib mampu menjelaskan setiap kelas dan elemen yang digunakan pada halamannya.
- Gambar, ikon, dan teks pada halaman produk harus milik sendiri, berlisensi bebas, atau disertai atribusi sesuai lisensinya.
- Kendala teknis disampaikan melalui Forum Tanya Jawab pada e-learning dengan menyertakan tangkapan layar dan pesan galat lengkap.

Daftar Pustaka

[1] Duckett, J. (2011). *HTML and CSS: Design and Build Websites*. Wiley.

[2] Mozilla. (2026). HTML elements reference. *MDN Web Docs*. https://developer.mozilla.org/en-US/docs/Web/HTML/Element

[3] Mozilla. (2026). CSS layout: Flexbox dan Grids. *MDN Web Docs*. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout

[4] Tailwind Labs. (2026). Responsive design dan Theme variables. *Tailwind CSS Documentation*. https://tailwindcss.com/docs/responsive-design

[5] World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

[6] W3C Web Accessibility Initiative. (2024). Landmark regions. *ARIA Authoring Practices Guide*. https://www.w3.org/WAI/ARIA/apg/practices/landmark-regions/

[7] Google. (2026). Lighthouse accessibility score. *Chrome for Developers*. https://developer.chrome.com/docs/lighthouse/accessibility/scoring

[8] Google. (2026). Simulate mobile devices with device mode dan Accessibility features reference. *Chrome for Developers*. https://developer.chrome.com/docs/devtools/device-mode

[9] Meta Open Source. (2026). Writing markup with JSX. *React Documentation*. https://react.dev/learn/writing-markup-with-jsx

[10] Vercel. (2026). Metadata and OG images. *Next.js Documentation*. https://nextjs.org/docs/app/getting-started/metadata-and-og-images